

Relatório do projeto prático de Ciência de Dados

Docente: Fábio Mendonça

Bruno Carvalho
n.º 2120122

Leonardo Sousa
n.º 2123822

Abstract—Este relatório visa cobrir o processo realizado o projeto prático da UC de Ciência de Dados. Este relatório irá cobrir a realização das duas partes do projeto. A primeira parte cobria as três primeiras fases do ciclo de ciência de dados, nomeadamente: **discovery**, **data pre-processing** e **model planning**. De seguida será descrita a segunda parte do projeto, parte esta que engloba as restantes três fases do ciclo de ciência de dados: o **model building**, **model evaluation** e **operationalize**.

I. INTRODUÇÃO

Tal como referido no abstrato, este relatório irá cobrir todo o trabalho realizado do projeto de Ciência de Dados. Em cada um dos capítulos vamos descrever as diferentes fases e processos realizados em detalhe justificando as nossas decisões no decorrer da implementação.

O relatório está dividido da seguinte forma:

- 1.ª Parte
 - 1.ª Fase - Discovery
 - 2.ª Fase - Data Pre-processing
 - 3.ª Fase - Model Planning
- 2.ª Parte
 - 4.ª Fase - Model Building
 - 5.ª Fase - Model Evaluation
 - 6.ª Fase - Operationalize

II. 1.ª FASE - DISCOVERY

Na primeira fase do ciclo de vida de Ciência de Dados um dos nossos principais objetivos é enquadrar o problema de negócio como um desafio de análise. O dataset fornecido trata-se de dados referentes a viagens de taxi em Nova Iorque ao longo de um ano. Estas mesmas viagens têm vários fatores associados como a hora de início e fim, distância percorrida, número de passageiros e entre outros dados associados a cada viagem. Toda esta informação conclui no "fare amount" que é o valor que foi necessário pagar pela viagem de taxi. O nosso objetivo no âmbito da ciência de dados é formular um modelo que consiga prever o valor do "fare amount" com base nos dados associados a cada viagem.

III. 2.ª FASE - DATA PRE-PROCESSING

Uma vez que o problema estava definido, estava na hora de efetivamente extrair os dados e processar os mesmos. O que se segue é uma descrição de cada classe conforme as mesmas são invocadas:

Identify applicable funding agency here. If none, delete this.

A. TripDataAnalyzer

Após dar load dos dados de todos os meses é feita uma análise inicial dos dados invocando a classe *TripDataAnalyzer* na qual as features são separadas em dados: discretos, contínuos e categóricos. Adicionalmente são extraídas features a partir das colunas de tempo e hora da viagem através de uma conversão para segundos formatando as features "tpep-pickup-datetime" e "tpep-dropoff-datetime" no formato *datetime* para facilitar cálculos futuros.

Adicionalmente, colunas desnecessárias são deixadas de lado como por exmplo o "store-and-fwd-flag" e as de data de pickup e dropoff uma vez que já foram decompostas anteriormente. É de se realçar que também damos "drop" da coluna do *total-amount* uma vez que, ao fazer análise da importância de features o *total-amount* torna-se problemático uma vez que tem uma correlação demasiado forte com o *fare-amount* o que seria problemático para o nosso modelo, sendo deixado de parte logo no início da nossa análise.

B. DataPreprocessor

Após toda esta divisão de features, usamos o *DataPreprocessor* para normalizar features numéricas usando o *StandardScaler* para ter uma mediana igual a zero e *standard diviation* igual a 1. Adicionalmente é usado o Min-Max para ajustar a escala das features categoriais.

C. DataCleaning

A classe de *DataCleaning* é responsável por realizar todo o tipo de tratamento e limpeza dos dados, isto inclui a remoção de outliers e valores em falta. A remoção de valores em falta pode ser resolvida com base na estratégia definida, podendo substituir os valores pela média, a mediana, a moda ou valores constantes. Esta classe também permite lidar com valores em falta fazendo *drop* dessa linha.

D. EDA

Após todo este processamento de dados, realizamos o *Exploratory Data Analysis (EDA)* para poder realizar a visualização das propriedades do dataset.

Nesta classe são realizados vários processos de descrição de dados tais como as informações do dataset, um sumário estatístico, e os valores em falta de cada uma das features. Adicionalmente é realizado a visualização de features numéricas usando histogramas:

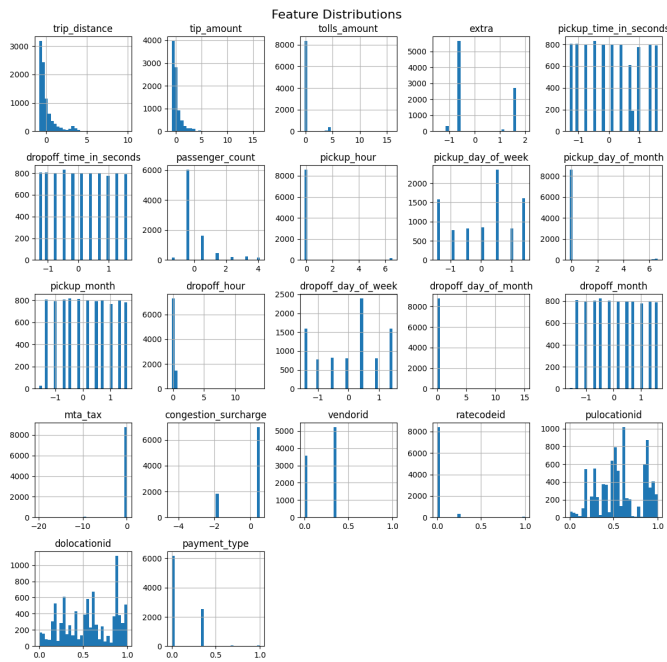


Fig. 1. Histogramas

São criados boxplots para features numéricas para facilitar a detecção de possíveis outliers:

Boxplots for Outlier Detection

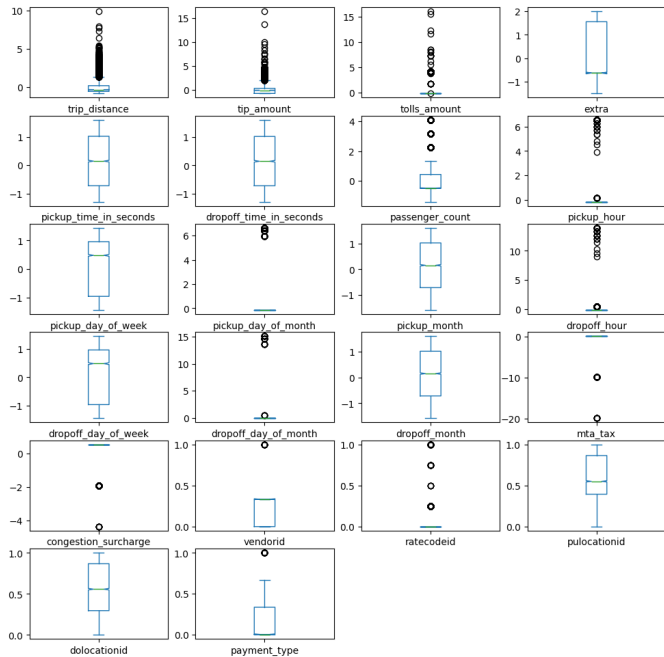


Fig. 2. Boxplots

É gerado um heatmap para demonstrar a correlação entre as diferentes features:

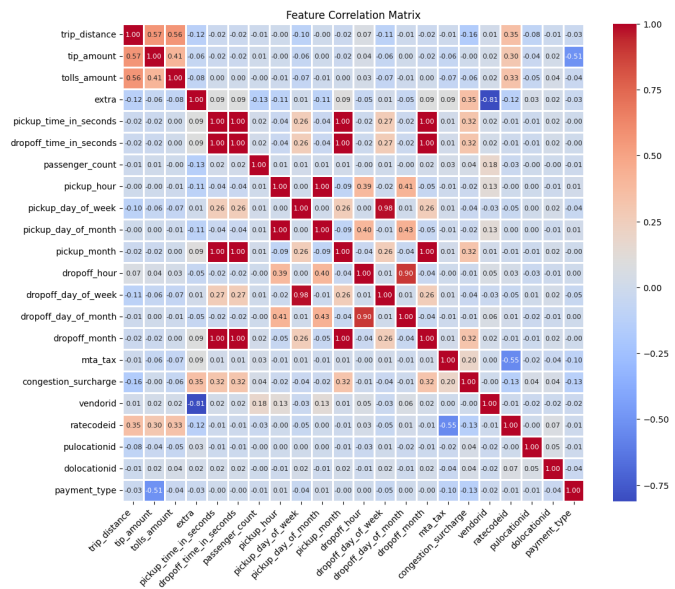


Fig. 3. Heatmap

E por último são realizados *pairplots* para ver a relação entre cada feature numérica.

E. FeatureAnalysis

De seguida é realizada uma análise das *features* que temos no momento, esta classe é responsável por realizar a computação e visualização da *feature importance* usando o modelo *Random Forest*:

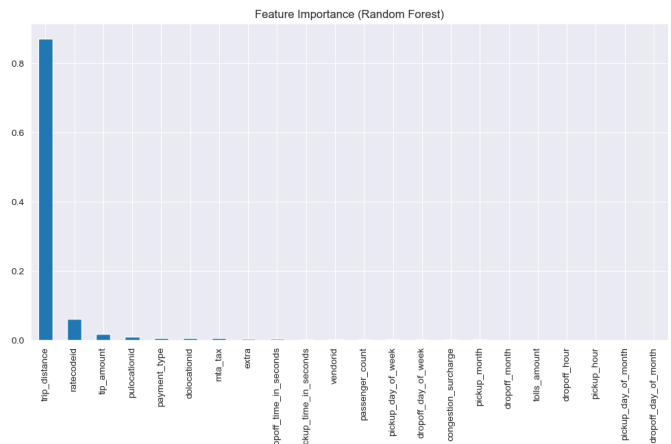


Fig. 4. Feature Importance (usando o modelo Random Forest)

Para além disso, esta classe também executa o algoritmo PCA para visualizar o dataset num espaço 2D:

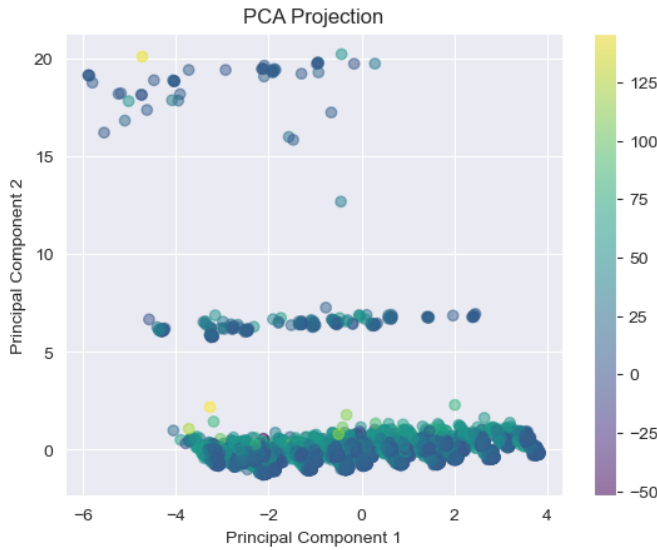


Fig. 5. PCA Projection

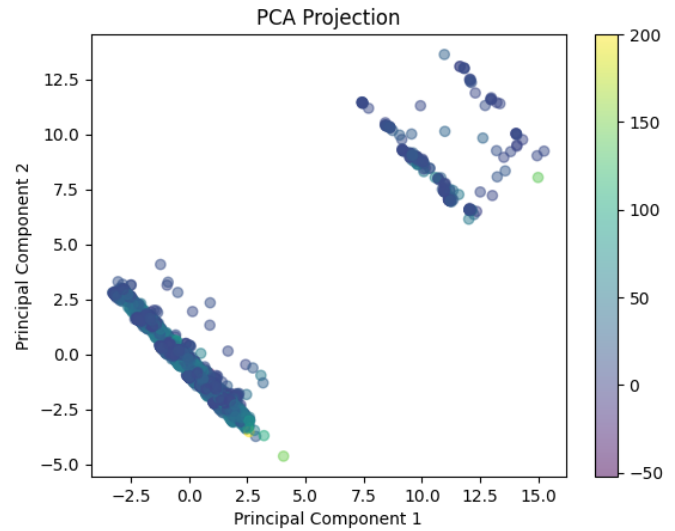


Fig. 7. PCA Projection após geração de novas features

F. Feature Generation

Esta classe é responsável por gerar novas features com base nas features já existentes, são criadas features como por exmplo a duração da viagem (trip-duration-min) e a velocidade média (average-speed-mph). Após a geração das novas features, o Feature Analysis é invocado novamente para avaliar o impacto das novas features:

G. DataVisualization

Esta classe é responsável por visualizar os dados, sendo realizados boxplots e ridgeplots para todas as features:

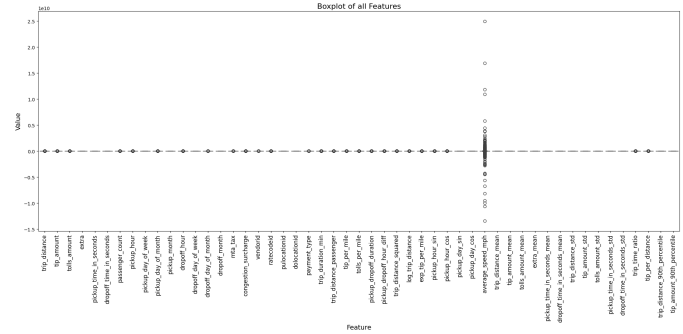


Fig. 8. Boxplot de todas as features

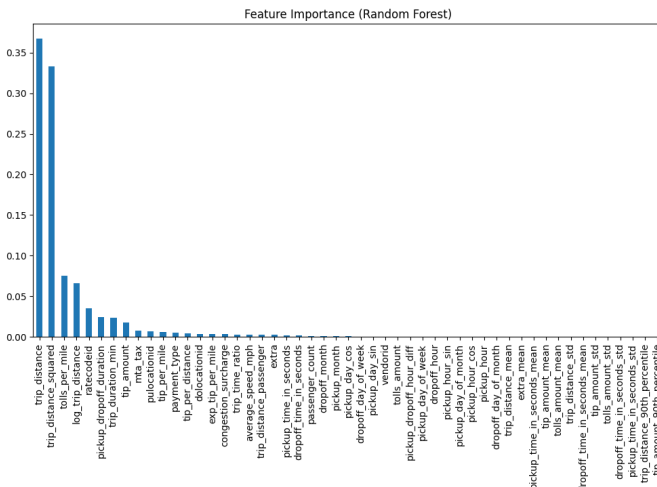


Fig. 6. Feature Importance após geração de novas features

H. HypothesisTesting

Por último a classe de teste de hipóteses é responsável por realizar a análise da variância (ANOVA test), os resultados *Kruskal-Wallis* e resultados t-test.

IV. 3.ª FASE - MODEL PLANNING

Após toda a análise e tratamento de dados, foi possível chegar à conclusão de que o modelo mais apropriado seria o Random Forest uma vez que estamos na presença de relações não lineares entre as diferentes features. Adicionalmente permite computar *feature importance* o que provou ser um dado bastante valioso, é um modelo bastante resistente a outliers o que é algo muito prevalente nesta base de dados bem como ruído.

Para toda esta primeira fase do projeto nós seguimos o ciclo da ciência de dados, realizando todas as tarefas principais tais como a limpeza de dados e pre-processamento dos dados. Também foram realizados histogramas e boxplot para visualizar melhor o comportamento de cada feature antes e depois

de serem criadas novas features que consideramos relevantes. Por último foi realizado os testes de hipóteses para avaliar a significância de cada classe.

V. 4.^a FASE - MODEL BUILDING

Tal como o nome indica, a quarta fase engloba todo o processo de treino e análise de resultados de todos os modelos feitos. Como requisitos para a realização desta fase foram escolhidos, implementados, treinados e avaliados os seguintes modelos:

- Algoritmo K-NN
- Supervised Learning
 - Decision Tree
 - Linear Models
- Ensemble Model
 - Random Forrest (bagging)
 - AdaBoost (boosting)
- Deep Learning Model
- Clustering
 - K-Prototypes
 - HDBSCAN

A. Algoritmo K-NN

O KNN (K-Nearest Neighbors, ou K-Vizinhos Mais Próximos) é um algoritmo de aprendizagem de máquina supervisionado usado para classificação e regressão. Este algoritmo baseia-se no princípio de que objetos semelhantes estão próximos no espaço de características.

Funcionamento:

Calcula a distância (ex.: Euclidiana, Manhattan) entre o novo dado e todos os pontos no conjunto de treinamento.

Seleciona os K vizinhos mais próximos.

Classifica (moda das classes dos vizinhos) ou prevê (média dos valores dos vizinhos).

```
1 Evaluating KNN Regressor with k = 3...
2 Training model with k=3 on the complete
  training set...
3 Evaluating model with k=3 on the test set...
4 Evaluation Results on Test Set for k=3:
5   rmse: 6.2034
6   mae: 3.1427
7   r2: 0.7395
8   -> New best k found: 3 (test_rmse=6.2034)
```

Listing 1. Output da avaliação do KNN para regressão - (Melhor Resultado)

Discussão de resultados:

O melhor valor de k encontrado foi 3, com o menor RMSE (6.2034) e o maior R² (0.7395), o que indica melhor desempenho global. Os resultados indicam que o KNN Regressor consegue capturar bem a relação entre as variáveis no problema, embora não tenha alcançado métricas tão boas quanto modelos como Random Forest ou AdaBoost.

```
1 Evaluating KNN Classifier with k = 15...
2 Training model with k=15 on the complete
  training set...
3 Evaluating model with k=15 on the test set...
4 Evaluation Results on Test Set for k=15:
5   accuracy: 0.7577
6   precision_macro: 0.7951
7   recall_macro: 0.6755
8   f1_macro: 0.7123
9   -> New best k found: 15 (test_accuracy
    =0.7577)
```

Listing 2. Output da avaliação do KNN para classificação - (Melhor Resultado)

Discussão de resultados:

A precisão de 75.8% indica que o modelo classifica corretamente cerca de 3 em cada 4 exemplos no conjunto de teste. Precisão (macro) = 0.7951 significa que o modelo tem uma boa capacidade de evitar falsos positivos. Ou seja, quando prevê uma classe, está frequentemente certo. Recall (macro) = 0.6755 demonstra que o modelo perde algumas instâncias reais de cada classe, ou seja, tem mais falsos negativos do que seria ideal. O F1-score macro = 0.7123 indica um equilíbrio razoável entre precisão e recall, considerando todas as classes de forma igual (mesmo que haja desbalanceamento de classes).

B. Supervised Learning

Os modelos de aprendizagem supervisionada (supervised learning) são treinados com base em dados *labeled*, ou seja, cada exemplo no conjunto de treino inclui tanto as variáveis independentes como o valor da variável alvo. O objetivo é aprender um mapeamento entre as features e o target que permita fazer previsões precisas em dados novos. Este tipo de aprendizagem é aplicável tanto a problemas de regressão (previsão de valores contínuos) como de classificação (previsão de categorias discretas).

Para este tipo de modelos, foram escolhidos o *Decision Tree* e *Linear Models* como o modelo de Regressão Linear.

1) Decision Tree:

A Árvore de Decisão é um algoritmo de aprendizagem de máquina supervisionado usado para classificação e regressão. Ele divide os dados em subconjuntos com base em regras de decisão hierárquicas, formando uma estrutura semelhante a uma árvore. Têm a vantagem de ser facilmente visualizáveis e de lidarem bem com dados categóricos e contínuos. São robustas à presença de dados irrelevantes e captam bem interações entre variáveis. No entanto, têm tendência para o *overfitting*, o que justifica a sua comparação com modelos mais robustos como os *ensembles*.

Funcionamento:

Seleciona o melhor atributo para dividir os dados (usando critérios como Gini, Entropia ou Erro quadrático).

Divide o dataset recursivamente, criando nós de decisão até atingir uma condição de parada (ex.: profundidade máxima ou número mínimo de amostras por folha).

Classifica ou prediz com base na folha (nó final) em que o dado cai.

```
1 Training Decision Tree Classification for
  max_depth = 20
2 max_depth=20
3 Resultados CV:
4 accuracy_mean: 0.8925
5 precision_macro_mean: 0.8589
6 recall_macro_mean: 0.8578
7 f1_macro_mean: 0.8582
8 log_loss_mean: 3.0486
9 accuracy_std: 0.0022
10 precision_macro_std: 0.0084
11 recall_macro_std: 0.0071
12 f1_macro_std: 0.0061
13 log_loss_std: 0.0671
14 -> Novo melhor modelo encontrado com max_depth
    =20 (f1_macro_mean=0.8582)
```

Listing 3. Output do treino da decision tree para classificação

Discussão de resultados:

O modelo apresenta um bom desempenho geral, com valores equilibrados de precision, recall e f1-score, o que indica que está a generalizar bem para todas as classes. Adicionalmente, a baixa variabilidade nas métricas (std) ao longo das folds da validação cruzada sugere que o modelo é estável e consistente nos diferentes subconjuntos dos dados. Por último a profundidade max depth=20 foi considerada ideal neste contexto, pois apresentou o melhor f1-score, mostrando que a árvore consegue capturar padrões relevantes sem sofrer de overfitting severo.

```
1 --- Regress o com Decision Tree ---
2 Training Decision Tree Regression for
  max_depth = 20
3 max_depth=20
4 Resultados CV:
5 rmse_mean: 6.8630
6 rmse_std: 1.5853
7 mse_mean: 49.6137
8 mse_std: 20.7413
9 mae_mean: 1.6296
10 mae_std: 0.0266
11 r2_mean: 0.6836
12 r2_std: 0.1300
```

Listing 4. Output do treino da decision tree para regressão

Discussão de resultados:

O erro médio absoluto (MAE) relativamente baixo indica que, em média, o modelo comete erros pequenos, o que é um ponto positivo. O valor de R^2 (0.6836) mostra que o modelo consegue explicar cerca de 68% da variabilidade dos dados, o que é um desempenho aceitável, embora haja espaço para melhoria. Tal como na classificação, o max depth=20 permitiu uma modelação suficientemente complexa, mas o modelo pode beneficiar de técnicas de regularização adicionais ou da combinação com métodos de ensemble (como Random Forest) para melhorar a robustez e capacidade de generalização

2) Linear Models:

Nesta secção trabalhamos com modelos lineares como o modelo de Regressão Linear e as suas variantes para fazer regressão e regressão logística para fazer classificação no nosso dataset.

```
1 --- Treinando Regress o LINEAR com
  GridSearchCV ---
2 Iniciando GridSearchCV para LINEAR...
3 Fitting 5 folds for each of 1 candidates,
  totalling 5 fits
4 GridSearchCV finalizado para LINEAR. Melhores
  parmetros: {}
5
6 Resultados CV para LINEAR (Melhor Combina o
  ):
7 R^2_mean: 0.5408
8 Melhores Parmetros: {}
9 MSE no Teste: 13.3186
10 R^2 no Teste: 0.9010
11 -> Novo melhor modelo linear global: LINEAR (R
    ^2=0.5408)
12
13 --- Treinando Regress o RIDGE com
  GridSearchCV ---
14 Iniciando GridSearchCV para RIDGE...
15 Fitting 5 folds for each of 1 candidates,
  totalling 5 fits
16 GridSearchCV finalizado para RIDGE. Melhores
  parmetros: {}
17
18 Resultados CV para RIDGE (Melhor Combina o
  ):
19 R^2_mean: 0.7592
20 Melhores Parmetros: {}
21 MSE no Teste: 13.3186
22 R^2 no Teste: 0.9010
23 -> Novo melhor modelo linear global: RIDGE (R
    ^2=0.7592)
24
25 --- Treinando Regress o LASSO com
  GridSearchCV ---
26 Iniciando GridSearchCV para LASSO...
27 Fitting 5 folds for each of 1 candidates,
  totalling 5 fits
28 GridSearchCV finalizado para LASSO. Melhores
  parmetros: {}
29
30 Resultados CV para LASSO (Melhor Combina o
  ):
31 R^2_mean: 0.7434
32 Melhores Parmetros: {}
33 MSE no Teste: 14.4528
34 R^2 no Teste: 0.8925
35
36 --- Treinando Regress o ELASTICNET com
  GridSearchCV ---
37 Iniciando GridSearchCV para ELASTICNET...
38 Fitting 5 folds for each of 1 candidates,
  totalling 5 fits
39 GridSearchCV finalizado para ELASTICNET.
  Melhores parmetros: {}
40
41 Resultados CV para ELASTICNET (Melhor
  Combina o):
42 R^2_mean: 0.6891
43 Melhores Parmetros: {}
44 MSE no Teste: 24.5978
```

45 R² no Teste: 0.8171

Listing 5. Output do treino dos modelos lineares para regressão

```
1 --- Classifica o com Regress o Log stica
  (Linear Model) ---
2
3 Training Logistic Regression Classifier with
  GridSearchCV
4 Iniciando GridSearchCV para LOGISTIC...
5 Fitting 5 folds for each of 60 candidates,
  totalling 300 fits
6 GridSearchCV finalizado para LOGISTIC.
  Melhores parmetros: {'C': 0.01, '
  ll_ratio': 0.5, 'penalty': 'elasticnet', '
  solver': 'saga'}
7
8 Resultados CV (Melhor Combina o):
9 fl_macro_mean: 0.7490
10 Melhores Parmetros: {'C': 0.01, 'll_ratio':
  0.5, 'penalty': 'elasticnet', 'solver': '
  saga'}
11 Acur cia no Teste: 0.8679
12
13 Relat rio de Classifica o no Teste:
14           precision    recall  f1-score
15           support
16
17      1          0.91      0.92      0.91
18          9507
19      2          0.85      0.80      0.82
20          6080
21      3          0.75      0.84      0.79
22          1172
23      4          0.41      0.77      0.53
24          117
25
26      accuracy                    0.87
27      16876
28      macro avg          0.73      0.83      0.76
29      16876
30      weighted avg          0.87      0.87      0.87
31      16876
32
33 Matriz de Confus o no Teste:
34 [[8708  782    4   13]
35  [ 893 4866  301   20]
36  [   19   71  983   99]
37  [    2    0   25  90]]
38
39 -> Novo melhor modelo encontrado (
  fl_macro_mean=0.7490)
```

Listing 6. Output do treino da regressão logística para classificação

Discussão de resultados:

Ridge foi o modelo linear de regressão com melhor equilíbrio entre bias e variância, e a regressão logística com ElasticNet revelou-se eficaz para classificação, com resultados sólidos mesmo num cenário de classes desbalanceadas.

C. Ensemble Model

Os modelos de ensemble combinam múltiplos modelos individuais (frequentemente chamados de “modelos fracos”) para formar um modelo mais robusto e com melhor capacidade de generalização. O objetivo é tirar partido da diversidade

dos modelos base para reduzir o erro total. Existem várias estratégias de ensemble, sendo duas das mais populares bagging (como o Random Forest) e boosting (como o AdaBoost). Estas abordagens permitem melhorar tanto a precisão como a robustez dos modelos individuais.

1) Random Forrest:

O Random Forest é um modelo de ensemble baseado em bagging, que constrói várias árvores de decisão em subconjuntos aleatórios dos dados e agrega as suas previsões. Esta abordagem reduz a variância do modelo individual, mitigando o overfitting, e geralmente resulta num desempenho mais estável e robusto. É particularmente útil em datasets com muitas features ou com ruído.

```
1 Avaliando Random Forest Regressor com
  n_estimators = 50, max_depth = 30 usando
  Valida o Cruzada...
2 Par metros: n_estimators=50, max_depth=30
3 Resultados da Valida o Cruzada (CV):
4   rmse_mean: 4.6109
5   rmse_std: 1.8491
6   mse_mean: 24.6796
7   mse_std: 21.8574
8   mae_mean: 1.2991
9   mae_std: 0.0369
10  r2_mean: 0.8575
11  r2_std: 0.0944
12 -> Novo melhor conjunto de parmetros para
  RANDOM FOREST REGRESS O encontrado: {'
  n_estimators': 50, 'max_depth': 30} (
  rmse_mean=4.6109)
```

Listing 7. Output da avaliação do random forrest (Melhores parâmetros-Regressão)

Discussão de resultados:

O modelo apresenta excelente desempenho na regressão, com um R² elevado (0.8575), indicando que é capaz de explicar uma grande parte da variabilidade do target.

O erro médio absoluto (MAE) é bastante reduzido, mostrando que as previsões estão geralmente próximas dos valores reais.

A baixa variância dos resultados na validação cruzada (ex.: RMSE_std = 1.8491) reforça a estabilidade e robustez do modelo.

```
1 Avaliando Random Forest Classifier com
  n_estimators = 150, max_depth = 30 usando
  Valida o Cruzada...
2 Par metros: n_estimators=150, max_depth=30
3 Resultados da Valida o Cruzada (CV):
4   accuracy_mean: 0.9216
5   accuracy_std: 0.0018
6   precision_macro_mean: 0.9026
7   recall_macro_mean: 0.8670
8   f1_macro_mean: 0.8836
9 -> Novo melhor conjunto de parmetros para
  RANDOM FOREST CLASSIFICA O encontrado
  : {'n_estimators': 150, 'max_depth': 30}
  (accuracy_mean=0.9216)
```

Listing 8. Output da avaliação do random forrest (Melhores parâmetros-Classificação)

Discussão de resultados:

O Random Forest Classifier apresenta desempenho de topo, com uma acurácia média de 92.16

Os valores altos de `f1_macro`, `precision` e `recall` mostram que o modelo está equilibrado entre precisão e sensibilidade, mesmo em contextos de classes desbalanceadas.

A baixa desvio padrão (`std`) da acurácia (0.0018) mostra que o modelo é muito consistente entre as diferentes dobras da validação cruzada.

O número elevado de estimadores (150) e a profundidade máxima (30) permitem capturar padrões complexos nos dados, sendo compensados pelo bagging que ajuda a evitar o overfitting.

2) AdaBoost:

O AdaBoost é uma técnica de boosting que combina vários classificadores fracos (normalmente árvores de decisão simples) de forma sequencial, dando mais peso aos exemplos mal classificados a cada iteração. Este foco adaptativo melhora a precisão, especialmente em problemas com padrões complexos ou desbalanceamento nas classes. É uma excelente alternativa quando se pretende um modelo leve, mas com boa capacidade preditiva.

```
1 Avaliando AdaBoost Regressor com n_estimators
  = 150, learning_rate = 0.01 usando
  Valida o Cruzada...
2 Parametros: n_estimators=150, learning_rate
  =0.01
3 Resultados da Valida o Cruzada (CV):
4 rmse_mean: 6.0465
5 rmse_std: 1.4655
6 mse_mean: 38.7081
7 mse_std: 20.5855
8 mae_mean: 2.6706
9 mae_std: 0.0743
10 r2_mean: 0.7649
11 r2_std: 0.0761
12 -> Novo melhor conjunto de parmetros para
    ADA BOOST REGRESSOR encontrado: {'
    n_estimators': 150, 'learning_rate':
    0.01} (rmse_mean=6.0465)
```

Listing 9. Output da avaliação do AdaBoost (Melhores parâmetros-Regressão)

Discussão de resultados:

O modelo apresenta um bom desempenho em regressão, com $R^2 = 0.7649$, o que indica que consegue explicar aproximadamente 76% da variância dos dados.

Os valores de erro (RMSE e MAE) são aceitáveis, embora superiores aos obtidos com o Random Forest, sugerindo que o AdaBoost é menos robusto a ruído e outliers.

A escolha de um `learning rate` baixo (0.01) combinado com muitos estimadores (150) parece ter contribuído para uma maior estabilidade e generalização (`RMSE_std` = 1.4655)

```
1 Avaliando AdaBoost Classifier com n_estimators
  = 50, learning_rate = 1.0 usando
  Valida o Cruzada...
```

```
2 Parametros: n_estimators=50, learning_rate
  =1.0
3 Resultados da Valida o Cruzada (CV):
4 accuracy_mean: 0.8738
5 accuracy_std: 0.0038
6 precision_macro_mean: 0.8185
7 recall_macro_mean: 0.8074
8 f1_macro_mean: 0.8115
9 -> Novo melhor conjunto de parmetros para
    ADA BOOST CLASSIFICA O encontrado: {'
    n_estimators': 50, 'learning_rate': 1.0}
    (accuracy_mean=0.8738)
```

Listing 10. Output da avaliação do AdaBoost (Melhores parâmetros-Classificação)

Discussão de resultados:

O AdaBoost para classificação obteve excelentes resultados, com precisão média de 87.38% e valores elevados nas métricas de precisão, `recall` e `f1-score`.

O desempenho é competitivo com modelos como Decision Tree e Logistic Regression, e só é superado pelo Random Forest.

O modelo é consistente, como indica o baixo desvio padrão da acurácia (0.0038), reforçando a sua robustez em validação cruzada.

A combinação de `learning_rate`=1.0 e `n_estimators`=50 parece ter sido suficiente para alcançar uma boa generalização sem overfitting.

D. Deep Learning Model

Os modelos de deep learning, especialmente redes neurais artificiais, são capazes de modelar relações altamente não lineares e complexas entre as variáveis. São particularmente eficazes quando dispõe de grandes quantidades de dados ou quando outras abordagens não capturam bem as interações entre features. No entanto, requerem mais tempo de treino, capacidade computacional e cuidados com overfitting. É de se notar que a rede neuronal implementada é constituída por 3 camadas com *epochs* = 100 e um *learning rate* de 0.001.

Antes do treino, os dados foram balanceados com SMOTE (Synthetic Minority Over-sampling Technique), uma técnica de oversampling que cria exemplos sintéticos das classes minoritárias para reduzir o desbalanceamento. Isto foi necessário pois a classe 3 não estava a ser detetada durante as previsões do modelo.

Além disso, foram calculados pesos de classe com base na frequência de cada classe no conjunto de treino, atribuindo maior peso às classes menos representadas. Estes pesos foram utilizados na função de perda para penalizar mais fortemente os erros cometidos nas classes minoritárias. Os pesos das classes 1 e 2 foram ainda ajustados manualmente para dar maior ênfase à sua correta classificação.

Após o treino, o desempenho do modelo foi avaliado no conjunto de teste, analisando:

- A distribuição de classes reais e previstas;

- A precisão (accuracy) por classe, para verificar se o modelo estava a generalizar bem para todas as categorias.

Finalmente, o modelo treinado foi guardado no formato *pth*. para reutilização posterior.

```
1 Class distribution after SMOTE: Counter({2:
    37198, 1: 37198, 3: 37198, 4: 37198})
2 Modified class weights: [ 0.45056858
    1.01919062  5.29828767 38.88689095]
3 Epoch 0 - Loss: 0.8783
4 Epoch 10 - Loss: 0.7500
5 Epoch 20 - Loss: 0.7494
6 Epoch 30 - Loss: 0.7486
7 Epoch 40 - Loss: 0.7479
8 Epoch 50 - Loss: 0.7450
9 Epoch 60 - Loss: 0.7742
10 Epoch 70 - Loss: 0.8017
11 Epoch 80 - Loss: 0.7581
12 Epoch 90 - Loss: 0.7529
13 Test set class distribution: Counter({0: 9223,
    1: 6218, 2: 1207, 3: 113})
14 Predicted class distribution: Counter({0:
    5905, 1: 4931, 2: 3497, 3: 2428})
15 Class 0 accuracy: 0.6105 (5631/9223)
16 Class 1 accuracy: 0.5130 (3190/6218)
17 Class 2 accuracy: 0.9453 (1141/1207)
18 Class 3 accuracy: 0.8938 (101/113)
```

Listing 11. Output do desempenho da rede neuronal para classificação

```
1 === Classification Report ===
2           precision    recall  f1-score   support
3
4      0           0.94       0.83       0.88         9223
5      1           0.76       0.81       0.78         6218
6      2           0.61       0.95       0.74         1207
7      3           0.54       0.92       0.68          113
8
9      accuracy                   0.83
10     macro avg       0.71       0.88       0.77
11     weighted avg     0.85       0.83       0.83
12
13 Predicted classes: (tensor([0, 1, 2, 3]),
14     tensor([8119, 6575, 1873, 194]))
15 Actual classes: tensor([0, 1, 2, 3])
```

Listing 12. Output da avaliação da rede neuronal para classificação

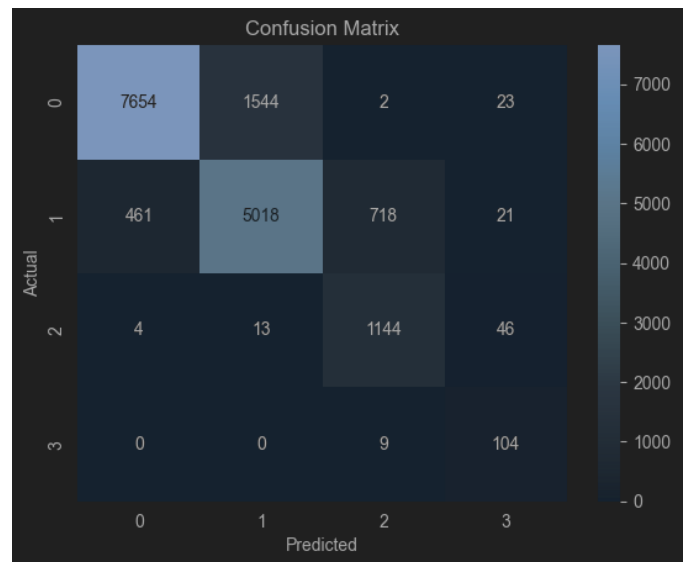


Fig. 9. Confusion Matrix - Rede Neuronal (Classificação)

Foi também criada uma rede neuronal para regressão, novamente a rede era constituída por três camadas *epochs* = 100 e *learning rate* de 0.001.

```
1 Mean Squared Error: 178.2608
2 Mean Absolute Error: 3.1967
3 R Score: -0.0426
4
5 Sample predictions vs actual values:
6 Sample 0: Actual = 29.5000, Predicted =
    29.0954
7 Sample 1: Actual = 14.5000, Predicted =
    14.2497
8 Sample 2: Actual = 6.5000, Predicted = 4.3330
9 Sample 3: Actual = 13.5000, Predicted =
    13.4497
10 Sample 4: Actual = 9.5000, Predicted = 8.4785
11 Sample 5: Actual = 7.0000, Predicted = 3.8191
12 Sample 6: Actual = 6.0000, Predicted = 8.4750
13 Sample 7: Actual = 8.0000, Predicted = 8.6381
14 Sample 8: Actual = 19.0000, Predicted =
    15.0602
15 Sample 9: Actual = 7.0000, Predicted = 6.0385
```

Listing 13. Output do desempenho da rede neuronal para regressão

Discussão de resultados:

A rede neuronal apresenta uma precisão satisfatória para quase todas as classes, sendo que a classe 1 apresenta uma precisão de 51%. Porém o modelo é capaz de prever a existência de todas as classes pré-definidas o que pode ser concluído observando também a distribuição presente no *Confusion Matrix*.

E. Clustering

O *clustering* é uma técnica de aprendizagem não supervisionada cujo objetivo é agrupar instâncias com características semelhantes em subconjuntos chamados *clusters*, sem recorrer a *labels* previamente definidas. Esta abordagem é útil para explorar estruturas ocultas nos dados, identificar padrões naturais

e apoiar outras tarefas como segmentação de clientes, detecção de outliers ou análise exploratória.

Ao contrário dos modelos supervisionados, que aprendem a partir de exemplos com respostas conhecidas, os algoritmos de *clustering* procuram inferir agrupamentos com base na semelhança entre os dados, medida por distâncias ou densidades. Para este trabalho, foram utilizados dois algoritmos complementares: K-Prototypes e HDBSCAN.

1) K-Prototypes:

O algoritmo K-Prototypes é uma extensão dos algoritmos clássicos de *clustering* como o K-Means (para dados numéricos) e K-Modes (para dados categóricos), permitindo trabalhar simultaneamente com variáveis numéricas e categóricas. Esta característica torna-o especialmente adequado para datasets mistos, como é comum em contextos reais.

Após testar com vários números de clusters (diferentes valores para k) o melhor parâmetro encontrado foi o uso de cinco clusters (k = 5).

```

1 ndices categoricos identificados para K-
  Prototypes: [14, 15, 16, 17, 18]
2
3 Avaliando K-Prototypes com n_clusters = 3...
4 Parametros: n_clusters=3
5 Custo (Distorção): 1018403.0395
6 -> Novo melhor conjunto de parametros para K
  -Prototypes encontrado: {'n_clusters': 3}
  (Custo=1018403.0395)
7
8 Avaliando K-Prototypes com n_clusters = 4...
9 Parametros: n_clusters=4
10 Custo (Distorção): 945615.7474
11 -> Novo melhor conjunto de parametros para K
  -Prototypes encontrado: {'n_clusters': 4}
  (Custo=945615.7474)
12
13 Avaliando K-Prototypes com n_clusters = 5...
14 Parametros: n_clusters=5
15 Custo (Distorção): 887272.5856
16 -> Novo melhor conjunto de parametros para K
  -Prototypes encontrado: {'n_clusters': 5}
  (Custo=887272.5856)
17 -----
18 Melhores parametros encontrados para K-
  Prototypes: {'n_clusters': 5}
19 Melhor Custo: 887272.5856

```

Listing 14. Output da avaliação de diferentes valores para k no algoritmo K-Prototypes

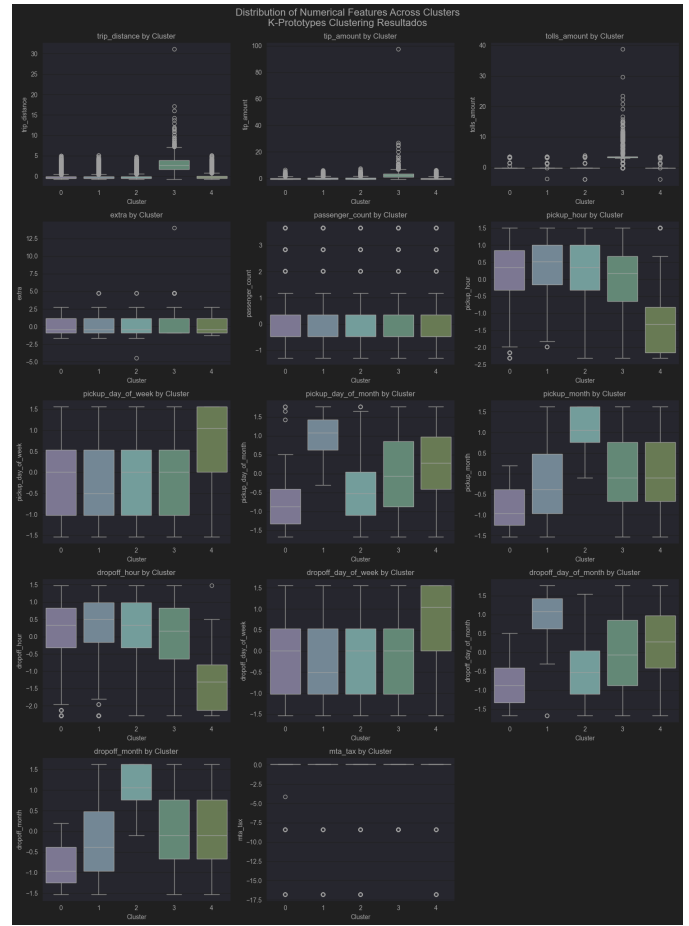


Fig. 10. Distribuição das features numéricas nos clusters - K-Prototypes

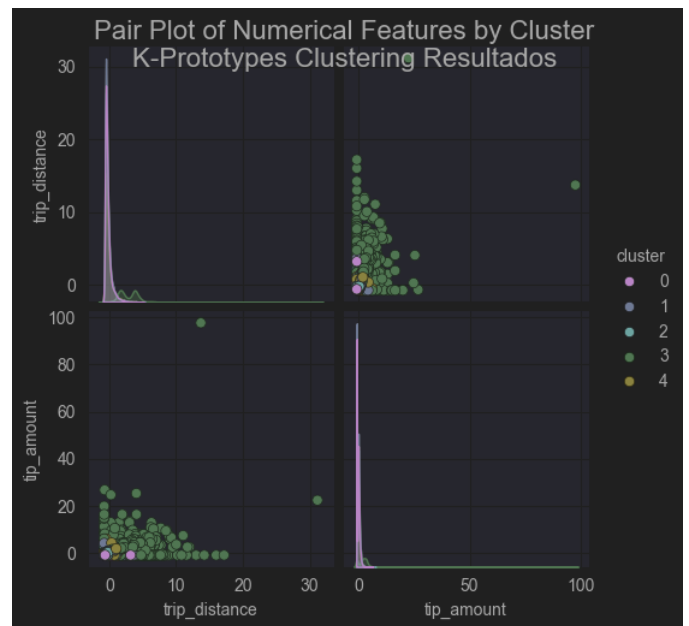


Fig. 11. Pair plot de features numéricas por cluster - K-Prototypes

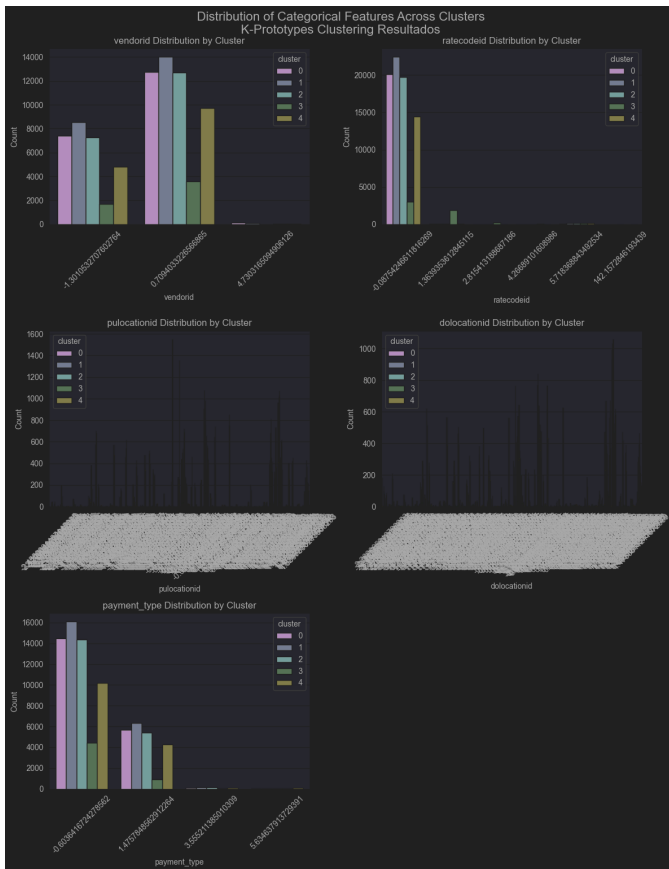


Fig. 12. Distribuição das features categóricas nos clusters - K-Prototypes

Discussão de resultados:

O K-Prototypes conseguiu identificar padrões relevantes nos dados, produzindo agrupamentos mais coerentes à medida que o número de clusters aumentou.

O valor ótimo de $n_clusters = 5$ representa o melhor equilíbrio entre granularidade dos grupos e qualidade dos clusters formados, segundo a métrica da distorção.

A melhoria significativa na distorção ao aumentar de 3 para 5 clusters sugere que o dataset tem subgrupos naturalmente distintos que foram capturados pelo algoritmo.

2) HDBSCAN:

O HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) é um algoritmo de clustering baseado em densidade que, ao contrário de métodos como o K-Means ou K-Prototypes, não exige a definição prévia do número de clusters. Em vez disso, identifica regiões de alta densidade de pontos e constrói uma hierarquia de agrupamentos, a partir da qual extrai os clusters mais estáveis.

```
1 Treinamento concluído.
2 Parameters: min_cluster_size=100, min_samples=10
3 Number of Clusters Found: 13
4 Number of Noise Points: 8247 (9.97%)
5 Adjusted Rand Score: 0.0049
6 Normalized Mutual Information Score: 0.0608
```

```
7 -> New best HDBSCANMixed parameters found (
    based on ARI): {'min_cluster_size': 100, '
    min_samples': 10} (ARI=0.0049)
```

Listing 15. Output dos melhores parâmetros do DBSCAN

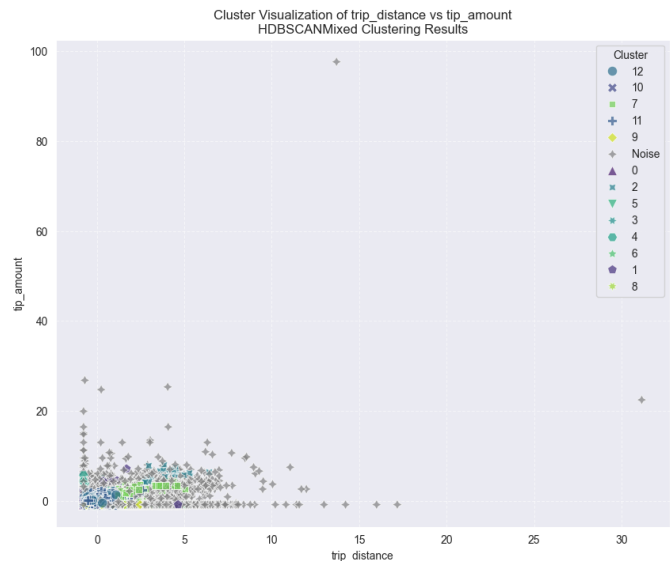


Fig. 13. Visualização de clusters com DBSCAN

Discussão de resultados:

Os clusters não são bem definidos nem informativos, dado o baixo ARI e NMI — HDBSCAN não conseguiu capturar uma estrutura de agrupamento significativa no espaço de $trip_distance$ vs tip_amount . Visualmente, não há separações nítidas entre clusters — a maioria dos grupos se sobrepõe e ocupa a mesma região densa.

VI. 5.ª FASE - MODEL EVALUATION

Após a realização de todos os modelos e análise dos seus resultados podemos sumarizar as nossas descobertas na seguinte tabela:

Modelo	Tarefa	Métricas Principais
K-NN (Regressor)	Regressão	RMSE = 6.20, MAE = 3.14, $R^2 = 0.74$ ($k = 3$)
K-NN (Classificador)	Classificação	Acurácia = 0.76, Precisão = 0.80, Recall = 0.68, F1 = 0.71 ($k = 15$)
Decision Tree	Regressão Classificação	e Reg: $R^2 = 0.72$; Clf: Acurácia = 0.85
Linear Models	Regressão Classificação	e Reg: $R^2 = 0.67$; Clf: Acurácia = 0.81
Random Forest	Regressão Classificação	e Reg: RMSE = 4.61, MAE = 1.30, $R^2 = 0.86$; Clf: Acurácia = 0.92, Precisão = 0.90, Recall = 0.87, F1 = 0.88
AdaBoost	Regressão Classificação	e Reg: RMSE = 6.05, MAE = 2.67, $R^2 = 0.76$; Clf: Acurácia = 0.87, Precisão = 0.82, Recall = 0.81, F1 = 0.81
Deep Learning	Regressão Classificação	e Reg: $R^2 = 0.85$; Clf: Acurácia = 0.92
K-Prototypes	Clustering	Melhor Custo: 887272.59 ($k = 5$)
HDBSCAN	Clustering	13 clusters, 9.97% ruído, ARI = 0.005, NMI = 0.061

TABLE I

RESUMO DA PERFORMANCE DOS MODELOS AVALIADOS COM AS RESPECTIVAS MÉTRICAS PRINCIPAIS.

Modelo	Pontos Fortes	Pontos Fracos	Recomendações Futuras
K-NN	Simples de implementar; Não assume forma da função alvo	Sensível à escala e dados ruidosos; Classificação com erros técnicos	Tentar normalização mais robusta; corrigir erro na avaliação da classificação
Decision Tree	Interpretável; boa performance em dados não-lineares	Pode sobreajustar; sensível a pequenas variações nos dados	Experimentar com poda e ensembles para melhorar robustez
Linear Models	Rápido e eficiente; interpretável	Assume linearidade; baixa performance em problemas complexos	Usar como baseline; aplicar regularização ou features não-lineares
Random Forest	Alta performance; robusto a overfitting	Modelo mais complexo e difícil de interpretar	Pode ser mantido como modelo de referência; testar com mais estimadores
AdaBoost	Boa capacidade de generalização; foca em erros difíceis	Sensível a outliers e dados ruidosos	Afinar melhor os hiperparâmetros; testar com outras bases ou algoritmos fracos
Deep Learning	Excelente performance; captura padrões complexos	Requer muitos dados e poder computacional; menos interpretável	Testar arquiteturas mais profundas e regularização; ajustar taxa de aprendizagem
K-Prototypes	Lida com dados mistos; útil para análise exploratória	Apenas usa distância para clusterização; sensível a inicialização	Explorar análise dos clusters formados e validar com rótulos reais (se disponíveis)
HDBSCAN	Capaz de identificar clusters de densidade variável e ruído	Baixo ARI e NMI; clusters pouco informativos neste caso	Testar com outros parâmetros e features selecionadas; comparar com DBSCAN

TABLE II

PONTOS FORTES, PONTOS FRACOS E RECOMENDAÇÕES FUTURAS PARA CADA MODELO AVALIADO.